Programlama Dilleri

Dr. Öğr. Üyesi Hayri Volkan Agun

Bilgisayar Mühendisliği Bölümü

Bursa Teknik Üniversitesi

Sözcüksel ve Sözdizimsel Analiz

- Derleyici tasarımının ciddi biçimde incelenmesi genellikle bir dönemlik yoğun çalışma ve küçük ama gerçekçi bir dil için derleyici geliştirmeyi gerektirir. Bu sürecin ilk aşaması **sözcüksel (lexical)** ve **sözdizimsel (syntax)** analizdir. Sözdizimsel analizör, anlamsal analiz ve ara kod üretimi gibi diğer bileşenleri yönlendirdiği için derleyicinin merkezidir.
- Programlama dilleri kitabında bu konunun yer almasının iki nedeni vardır:
- Sözdizimsel analizörler doğrudan dilbilgilerine dayanır; bu nedenle dilbilgilerinin doğal bir uygulamasıdır.
- Sözcüksel ve sözdizimsel analiz yalnızca derleyicilerde değil; kod biçimlendiriciler, karmaşıklık analiz araçları ve yapılandırma dosyalarını işleyen sistemler gibi birçok yazılımda da gereklidir.

Derleme

- Programlama dillerinin gerçekleştirilmesinde üç temel yaklaşım vardır:
- **Derleme (compilation):** Yüksek seviyeli dili makine koduna çeviren derleyiciler kullanılır. Büyük ve performans kritik uygulamalarda (ör. C++, COBOL) yaygındır.
- **Saf yorumlama (pure interpretation):** Çeviri yapılmaz; program doğrudan yorumlayıcı tarafından çalıştırılır. Verimliliğin kritik olmadığı durumlarda, örneğin HTML içine gömülü betiklerde (örn. JavaScript) kullanılır.
- **Hibrit yaklaşım:** Programlar önce ara koda çevrilir, ardından bu ara biçim yorumlanır.

Sözcüksel analiz

- Sözcüksel analiz (lexical analysis) ile sözdizimsel analiz (syntax analysis) üç temel nedenle ayrılır:
- **Basitlik:**
  Sözcüksel analiz teknikleri, sözdizimsel analiz tekniklerine göre daha az karmaşıktır. Ayrı bir bileşen olarak tasarlanması hem süreci sadeleştirir hem de sözdizimsel analizörün daha küçük ve anlaşılır olmasını sağlar.
- **Verimlilik:**
  Derleme süresinin önemli bir kısmı sözcüksel analizde harcanır; bu nedenle bu aşamayı optimize etmek anlamlıdır. Buna karşılık sözdizimsel analizde agresif optimizasyon genellikle fayda sağlamaz. Ayrım, seçici optimizasyona imkân tanır.
- **Taşınabilirlik:**
  Sözcüksel analizör, kaynak dosyaları okur ve çoğu zaman tamponlama (buffering) içerir; bu nedenle platforma bağımlıdır. Sözdizimsel analizör ise büyük ölçüde platformdan bağımsız olabilir. Makineye bağımlı bileşenlerin izole edilmesi yazılım mühendisliği açısından tercih edilir.

Sözcüksel Analiz

- Bir **sözcüksel analizör (lexical analyzer)**, sözdizimsel analizörün (syntax analyzer) ön yüzü olarak çalışır. Teknik olarak, sözcüksel analiz sözdizimsel analizin en düşük seviyesidir.
- Girdi programı derleyiciye tek bir karakter dizisi olarak görünür. Sözcüksel analizör bu karakterleri mantıksal gruplar hâline getirir ve gruplara yapılarına göre **iç kodlar (internal codes)** atar.
- Bu mantıksal gruplara **lexeme** denir. Lexem türlerinin iç kodlarına **token** denir.
- Lexemler, giriş karakter dizisi ile karakter deseni (pattern) eşleştirilerek tanınır. Tokenlar genellikle tamsayı olarak temsil edilir; ancak okunabilirlik için genellikle **isimlendirilmiş sabitler (named constants)** üzerinden referans verilir.
- Atama ifadesi:

  result = oldsum - value / 100;

- Bu ifadenin **lexem** ve **token** karşılıkları şu şekilde olur...

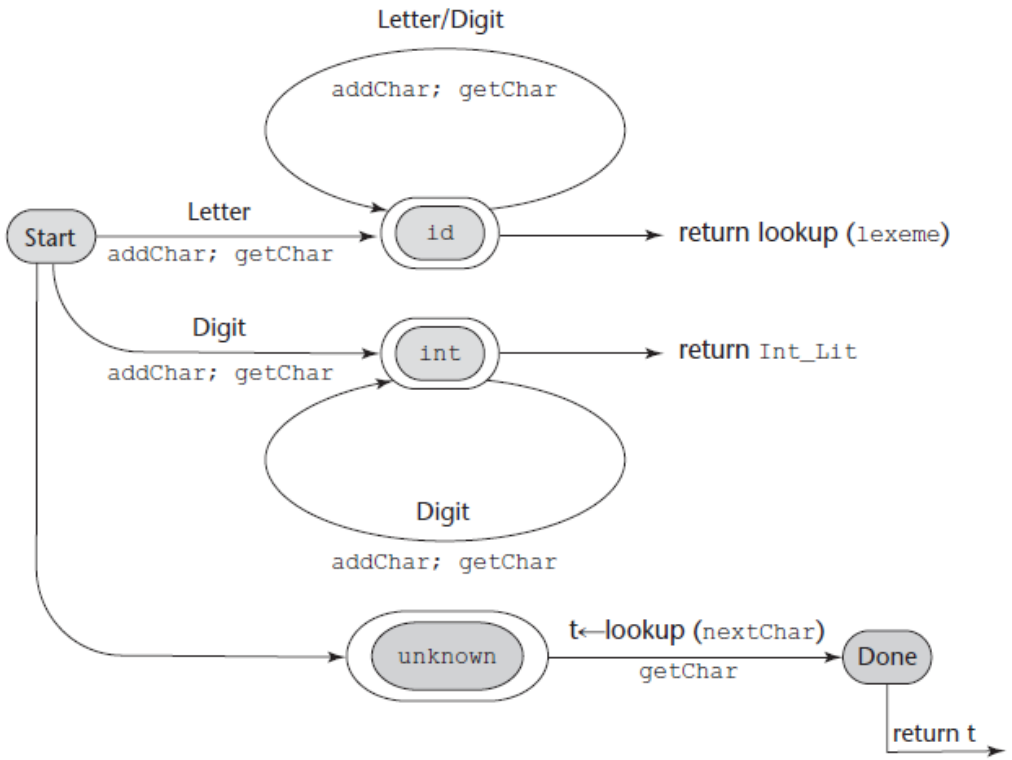
Token	*Lexeme*
IDENT	result
ASSIGN_OP	=
IDENT	oldsum
SUB_OP	-
IDENT	value
DIV_OP	/
INT_LIT	100
SEMICOLON	;

Sözcüksel Analiz

- Sözcüksel analizör (lexical analyzer) oluşturmanın üç temel yaklaşımı vardır:
- **Otomatik üretim:** Dilin token desenlerini tanımlayan formal bir betimleyici (regular expression tabanlı) dil kullanılır. Bu tanımlar bir yazılım aracına verilir ve araç otomatik olarak bir sözcüksel analizör üretir. Bu amaçla birçok araç mevcuttur; en eskisi ve yaygın olanı **lex**, UNIX sistemlerinde sıklıkla bulunur.
- **Durum diyagramı ile programlama:** Dilin token desenlerini tanımlayan bir **durum geçiş diyagramı (state transition diagram)** tasarlanır ve bu diyagramı uygulayan bir program yazılır.
- **Tablo tabanlı elle uygulama:** Durum diyagramı tasarlanır ve bu diyagramın tablo tabanlı bir uygulaması elle oluşturulur.

```c
/* front.c - a lexical analyzer system for simple
             arithmetic expressions */

#include <stdio.h>
#include <ctype.h>

/* Global declarations */
/* Variables */
int  charClass;
char  lexeme [100];
char  nextChar;
int  lexLen;
int  token;
int  nextToken;
FILE *in_fp, *fopen();

/* Function declarations */
void  addChar();
void  getChar();
void  getNonBlank();
int  lex();

/* Character classes */
#define LETTER 0
#define DIGIT 1
#define UNKNOWN 99

/* Token codes */
#define INT_LIT 10
#define IDENT 11
#define ASSIGN_OP 20
#define ADD_OP 21
#define SUB_OP 22
#define MULT_OP 23
#define DIV_OP 24
#define LEFT_PAREN 25
#define RIGHT_PAREN 26
```

Sözcüksel Analiz




```
(sum + 47) / total

Next token is: 25 Next lexeme is (
Next token is: 11 Next lexeme is sum
Next token is: 21 Next lexeme is +
Next token is: 10 Next lexeme is 47
Next token is: 26 Next lexeme is )
Next token is: 24 Next lexeme is /
Next token is: 11 Next lexeme is total
Next token is: -1 Next lexeme is EOF
```

Sözdizimsel Analiz - Ayrıştırma

- Programlama dili **çözücüler (parsers)**, verilen programlar için **çözümleme ağaçları (parse trees)** oluşturur.
- Bazı durumlarda ağaç yalnızca **örtük olarak** oluşturulur; yani ağaç doğrudan tutulmaz, yalnızca üzerinde gezinme (traversal) yapılır.
- Ancak her durumda, çözümleme sırasında ağacı oluşturmak için gereken tüm bilgi üretilir.
- **Çözümleme ağaçları ve türetimler (derivations)**, bir dil işlemcisinin ihtiyaç duyduğu tüm **sözdizimsel bilgileri** içerir.
- Sözdizimsel analizin iki farklı amacı vardır: İlk olarak, sözdizimsel analizör, girdi programının sözdizimsel olarak doğru olup olmadığını kontrol etmelidir. Bir hata bulunduğunda, analizör bir tanı mesajı üretmeli ve hatayı düzeltmelidir.
- Bu durumda düzeltme, normal bir duruma geri dönmesi ve girdi programının analizine devam etmesi anlamına gelir. Bu adım, derleyicinin girdi programının tek bir analizinde mümkün olduğunca çok hata bulabilmesi için gereklidir. Eğer bu iyi yapılmazsa, hata düzeltme daha fazla hata veya en azından daha fazla hata mesajı üretebilir. Sözdizimsel analizin ikinci amacı, sözdizimsel olarak doğru girdiler için tam bir çözümleme ağacı üretmek veya en azından tam çözümleme ağacının yapısını izlemektir. Çözümleme ağacı (veya izlediği yol) çeviri için temel olarak kullanılır.
- Çözücüler, çözümleme ağaçlarını oluşturma yönlerine göre sınıflandırılır. Çözücülerin iki ana sınıfı vardır:
- **Kökten-yapraklara (top-down)**: Ağaç kökten yapraklara doğru inşa edilir.
- **Yapraktan-köke (bottom-up)**: Çözümleme ağacı yapraklardan köke doğru inşa edilir.

Yukarıdan Aşağıya Ayrıştırma

- Türetilme açısından, bir **kökten-yapraklara (top-down) çözücü** şu şekilde tanımlanabilir: Sol en soldan türetme (leftmost derivation) içinde yer alan bir **cümsel form** (sentential form) verildiğinde, özücünün görevi bu sol en soldan türetmede bir sonraki cümsel formu bulmaktır.
- Bir sol cümsel formun genel biçimi **xAa** şeklindedir: x terminal sembollerden oluşan bir dizidir, A bir **nonterminal**, a karışık bir dizi (terminal ve/veya nonterminal) içerir. Çünkü x yalnızca terminallerden oluşur, A bu cümsel formdaki **en soldaki nonterminal**dir. Dolayısıyla bir sonraki cümsel formu elde etmek için A genişletilmelidir.
- Bir sonraki cümsel formu belirlemek, A'yı sol taraf (LHS) olarak içeren doğru gramer kuralını seçmekle ilgilidir. Örnek: xAa

  A kuralları: ASbB, AScBb, ASa
- Bir top-down çözücü, bu üç kuraldan birini seçerek bir sonraki cümsel formu elde eder: xbBa, xcBba, veya xaa
- İşte bu, **top-down çözücüler için çözümleme kararı problemi (parsing decision problem)** olarak adlandırılır.

Aşağıdan Yukarı Ayrıştırma

- Bir **yapraktan-köke (bottom-up) çözücü**, çözümleme ağacını yapraklardan başlatarak köke doğru oluşturur. Bu çözümleme sırası, **sağ en soldan türetmenin (rightmost derivation) tersine** karşılık gelir; yani türetmedeki cümsel formlar sonundan başına doğru üretilir.
- Türetilme açısından bir bottom-up çözücü şöyle tanımlanabilir:
- Verilen bir **sağ cümsel form (right sentential form) α** için, çözücü, türetmedeki bir önceki cümsel formu elde etmek amacıyla α içinde hangi alt dizinin gramerdeki kuralın sağ tarafı (RHS) olduğunu belirlemelidir ve bunu sol tarafla (LHS) değiştirmelidir.

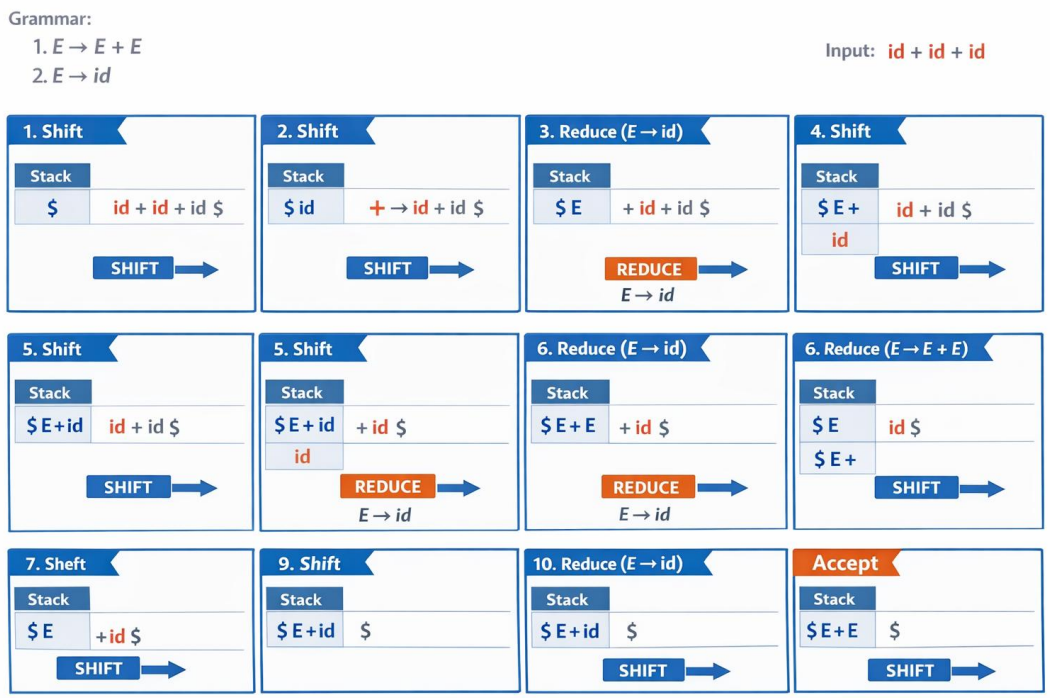
  S → SaAc
  A → aA | b

- Türetme: S => aAc => aaAc => aabc

- Bottom-up çözücü, cümle aabc ile başlar ve içinde **handle** olarak adlandırılan RHS'yi bulur. Bu örnekte b tek RHS olduğu için kolaydır. b LHS olan A ile değiştirildiğinde, türetmedeki sondan bir önceki cümsel form olan aaAc elde edilir.
- Genel durumda, handle'ı bulmak daha zordur çünkü bir sağ cümsel form birden fazla RHS içerebilir.
- Çözücü, olası handle'ın bir veya iki yanındaki sembolleri inceleyerek doğru RHS'yi bulur.
- Handle'ın sağındaki semboller genellikle hâlâ analiz edilmemiş giriş tokenlarıdır.
- En yaygın bottom-up çözümleme algoritmaları **LR ailesindendir**:
- L girişin soldan-sağa tarandığını,
- R sağ en soldan türetme üretildiğini belirtir.

Aşağıdan Yukarı Ayrıştırma

- **Bottom-up çözücüler** genellikle **shift-reduce algoritmaları** olarak adlandırılır, çünkü en yaygın iki işlem **shift** ve **reduce**dur.
- Her bottom-up çözücünün temel bileşeni bir **yığın (stack)**tır.
- Girdi: programın **token akışı**
- Çıktı: **gramer kuralları dizisi**
- **İşlemler**
- **Shift:** Bir sonraki girdi tokenını çözücünün yığınına taşır.
- **Reduce:** Yığının tepesindeki RHS (handle) LHS ile değiştirilir.



Soldan Azaltma

- Bir **LR çözümleme tablosu**, iki bölümden oluşur: **ACTION** ve **GOTO**.
- **1. ACTION Bölümü**
- Çözümleyicinin yaptığı işlemlerin çoğunu belirler.
- Satırlar: **durum sembolleri (state symbols)**
- Sütunlar: gramerin **terminal sembolleri**
- Kullanımı:
  - Parse yığınının tepesindeki durum sembolü ve girişteki bir sonraki token verildiğinde, tablo çözümleyicinin ne yapacağını belirtir.
  - **Ana işlemler:**
    - **Shift:** Bir sonraki giriş sembolü yığına eklenir.
    - **Reduce:** Yığının tepesindeki handle, RHS ile eşleşen kuralın LHS'si ile değiştirilir.
  - **Diğer işlemler:**
    - **Accept:** Giriş başarıyla çözümlendi.
    - **Error:** Sözdizimsel hata bulundu.
- **2. GOTO Bölümü**
- Satırlar: **durum sembolleri**
- Sütunlar: **nonterminal semboller**
- Kullanımı:
  - Bir **reduction** tamamlandıktan sonra (handle yığından çıkarıldı ve yeni nonterminal yığına itildiğinde), GOTO tablosu hangi durum sembolünün yığına eklenmesi gerektiğini gösterir.
  - Satır: yığının tepesindeki durum sembolü (handle ve ilişkili durum sembolleri çıkarıldıktan sonra)
  - Sütun: kuralın LHS'si (reduction yapılan kuralın sol tarafı)
- Bu iki tablo birlikte, LR çözümleyicinin doğru şekilde shift, reduce, accept veya error kararlarını vermesini sağlar ve yığındaki durumları günceller.

Soldan Azaltma

$E \rightarrow E + T$

$E \rightarrow T$

$T \rightarrow T * F$

$T \rightarrow F$

$F \rightarrow (E)$

$F \rightarrow id$

	Action						Goto		
State	id	+	*	(	)	$	E	T	F
0	S5			S4			1	2	3
1		S6				accept			
2		R2	S7		R2	R2			
3		R4	R4		R4	R4			
4	S5			S4			8	2	3
5		R6	R6		R6	R6			
6	S5			S4				9	3
7	S5			S4					10
8		S6			S11				
9		R1	S7		R1	R1			
10		R3	R3		R3	R3			
11		R5	R5		R5	R5			

Soldan Azaltma

Stack	*Input*	*Action*
0	id + id * id $	Shift 5
0id5	+ id * id $	Reduce 6 (use GOTO[0, F])
0F3	+ id * id $	Reduce 4 (use GOTO[0, T])
0T2	+ id * id $	Reduce 2 (use GOTO[0, E])
0E1	+ id * id $	Shift 6
0E1+6	id * id $	Shift 5
0E1+6id5	* id $	Reduce 6 (use GOTO[6, F])
0E1+6F3	* id $	Reduce 4 (use GOTO[6, T])
0E1+6T9	* id $	Shift 7
0E1+6T9*7	id $	Shift 5
0E1+6T9*7id5	$	Reduce 6 (use GOTO[7, F])
0E1+6T9*7F10	$	Reduce 3 (use GOTO[6, T])
0E1+6T9	$	Reduce 1 (use GOTO[0, E])
0E1	$	Accept

	Action						Goto		
State	id	+	*	(	)	$	E	T	F
0	S5			S4			1	2	3
1		S6				accept			
2		R2	S7		R2	R2			
3		R4	R4		R4	R4			
4	S5			S4			8	2	3
5		R6	R6		R6	R6			
6	S5			S4				9	3
7	S5			S4					10
8		S6			S11				
9		R1	S7		R1	R1			
10		R3	R3		R3	R3			
11		R5	R5		R5	R5			

Soldan Azaltma

- **Adım 1: Başlangıç**
- Başlangıç durumu: **0**
- Yığın (stack): [0]
- Girdi: id + id * id $
- **Adım 2: İlk Shift**
- Yığının tepesindeki durum: 0
- Girdi sembolü: id
- ACTION[0, id] = S5 → shift 5
- Yığındaki değişiklik: [0, id, 5]
- Girdi: + id * id $
- **Adım 3: Reduce**
- Yığının tepesindeki durum: 5
- Girdi sembolü: +
- ACTION[5, +] = R6 → Reduce kuralı 6
- Örneğin kural 6: F → id
- Reduce işlemi: id LHS olan F ile değişir
- Yığındaki değişiklik: [0, F, 3] (3 GOTO[0, F] = 3)
- Girdi: + id * id $
- **Adım 4: Shift +**
- Yığının tepesindeki durum: 3
- Girdi sembolü: +
- ACTION[3, +] = R4 → Reduce kuralı 4
- Örneğin kural 4: T → F
- Yığındaki değişiklik: [0, T, 2] (2 GOTO[0, T] = 2)

Soldan Azaltma

- **Adım 5: Shift +**
- Yığının tepesindeki durum: 2
- Girdi sembolü: +
- ACTION[2, +] = S7 → shift 7
- Yığındaki değişiklik: [0, E, 1, +, 7]
- Girdi: id * id $
- **Adım 6: Shift id**
- Yığının tepesindeki durum: 7
- Girdi sembolü: id
- ACTION[7, id] = S5 → shift 5
- Yığındaki değişiklik: [0, E, 1, +, 7, id, 5]
- Girdi: * id $
- **Adım 7: Reduce id → F**
- Yığının tepesindeki durum: 5
- Girdi sembolü: *
- ACTION[5, *] = R6 → Reduce kuralı 6: F → id
- Yığındaki değişiklik: [0, E, 1, +, 7, F, 10] (10 GOTO[7, F] = 10)
- Girdi: * id $
- **Adım 8: Shift ***
- Yığının tepesindeki durum: 10
- Girdi sembolü: *
- ACTION[10, *] = S7 → shift 7
- Yığındaki değişiklik: [0, E, 1, +, 7, F, 10, *, 7]
- Girdi: id $
- **Adım 9: Shift id → Reduce → Devam**
- Bu süreç tekrarlanır: id → F reduce, sonra F → T reduce, ardından T → E reduce.
- Sonunda girişin tamamı işlenir ve tablo ACTION[1, $] = **accept** → parse başarıyla tamamlanır.